Almond: a GPU spherical-harmonic transform for HEALPix

Mathematical techniques and technical design

PowerSpec / SiMaster project

July 2026 (v0.4: spin-0 and spin-2, synthesis and exact adjoint; originally prototyped for spin-0 synthesis in v0.1)

Abstract

Almond is a CUDA implementation of the spherical-harmonic transform (SHT) on HEALPix grids, intended to replace the CPU library `ducc0` as the SHT backend of the SiMaster quadratic maximum-likelihood (QML) power spectrum estimator. It implements, on the GPU, the same exact algorithm as `ducc0` (the libsharp2 lineage): double precision throughout, healpy conventions, matrix-free, with the every-other-ℓ Legendre recursion, $2^{\pm 800}$ dynamic-range scaling, and per-ring FFT-bin aliasing. It covers spin-0 and spin-2 fields in both directions (synthesis and exact adjoint synthesis). `ducc0` is the correctness reference; Almond achieves $\lesssim 10^{-11}$ maximum relative deviation from it. This document derives the algorithm, records the design decisions and their GPU-specific rationale, and reports benchmark results on NERSC Perlmutter A100 nodes.

Contents

1 Problem statement and conventions

A real spin-0 field sampled on a HEALPix RING grid of resolution $N_{\rm side}$ is synthesised from harmonic coefficients as

$$f(\theta,\varphi) \;=\; \sum_{\ell=0}^{\ell_{\max}} \sum_{m=-\ell}^{\ell} a_{\ell m}\, Y_{\ell m}(\theta,\varphi), \qquad Y_{\ell m}(\theta,\varphi) = \lambda_{\ell m}(\theta)\, e^{im\varphi}, \tag{1}$$

where $\lambda_{\ell m}$ are the fully normalised associated Legendre functions ($\int |Y_{\ell m}|^2 d\Omega = 1$). healpy stores only $m \geq 0$; reality implies $a_{\ell,-m} = (-1)^m a^*_{\ell m}$ and $\lambda_{\ell,-m} = (-1)^m \lambda_{\ell m}$. The triangular healpy layout indexes $a_{\ell m}$ at $\mathrm{mstart}(m) + \ell$ with $\mathrm{mstart}(m) = m(2\ell_{\max} + 1 - m)/2$. Following `ducc0`/healpy, the imaginary part of $a_{\ell 0}$ is ignored.

HEALPix RING geometry places pixels on $4N_{\rm side}-1$ iso-latitude rings; ring i ($1 \leq i \leq 4N_{\rm side}-1$, north to south) has

$$n_\varphi(i) = \begin{cases} 4i' & \text{polar caps } (i' < N_{\rm side}) \\ 4N_{\rm side} & \text{equatorial belt} \end{cases} \qquad \varphi_j = \varphi_0(i) + \frac{2\pi j}{n_\varphi}, \tag{2}$$

with $i' = \min(i, 4N_{\rm side} - i)$, and $\cos\theta$ an exact rational: $1 - i'^2/(3N_{\rm side}^2)$ in the caps, $4/3 - 2i/(3N_{\rm side})$ in the belt. We compute $\cos\theta$, $\sin\theta$ directly from these rationals (never through $\theta = \arccos z$), and $\varphi_0 = \pi q/d$ with small integers (q, d) — both facts are exploited for accuracy (§3).

The transform factorises over rings:

$$F_m(\theta) \;=\; \sum_{\ell=m}^{\ell_{\max}} a_{\ell m}\, \lambda_{\ell m}(\theta), \qquad f(\theta,\varphi_j) \;=\; \sum_{m=-\ell_{\max}}^{\ell_{\max}} F_m(\theta)\, e^{im\varphi_j}, \quad F_{-m} = F_m^*. \tag{3}$$

Stage 1 (*Legendre*, $O(N_{\rm ring}\, \ell_{\max}^2)$ work) computes F_m on every ring; stage 2 (*FFT*, $O(N_{\rm pix} \log N_{\rm side})$) evaluates the azimuthal sums. Stage 1 dominates at all resolutions of interest.

2 Stage 1: the every-other-ℓ Legendre recursion

2.1 Three-term recurrence and ring pairing

The normalised Legendre functions obey

$$\cos\theta\, \lambda_{\ell m} = \varepsilon_{\ell+1} \lambda_{\ell+1,m} + \varepsilon_\ell \lambda_{\ell-1,m}, \qquad \varepsilon_\ell = \sqrt{\frac{\ell^2 - m^2}{4\ell^2 - 1}}, \tag{4}$$

seeded by $\lambda_{mm}(\theta) = c_m \sin\theta^m$, $c_m = (-1)^m \sqrt{(2m+1)!!/(4\pi\,(2m)!!)}$, and $\lambda_{m-1,m} = 0$. The parity relation $\lambda_{\ell m}(\pi - \theta) = (-1)^{\ell+m} \lambda_{\ell m}(\theta)$ makes it natural to process rings in north/south pairs sharing $|\cos\theta|$: splitting the sum in (3) into even and odd $\ell - m$,

$$S_e = \sum_{\ell-m \text{ even}} a_{\ell m} \lambda_{\ell m}, \qquad S_o = \sum_{\ell-m \text{ odd}} a_{\ell m} \lambda_{\ell m}, \qquad F_m^{\rm north} = S_e + S_o, \quad F_m^{\rm south} = S_e - S_o. \tag{5}$$

This halves the Legendre work. The HEALPix equator ring is self-paired ($\cos\theta = 0$).

2.2 Folding the odd ℓ into the even stream

`ducc0`'s key trick (inherited here) is to visit only every other ℓ in the θ recursion. Write $n_i = m+2i+1$ for the odd offsets, $x = \cos^2\theta$. Applying (4) twice gives a pure $\Delta\ell = 2$ recurrence,

$$\lambda_{n+2} \;=\; \frac{(x - \varepsilon_n^2 - \varepsilon_{n+1}^2)\,\lambda_n \;-\; \varepsilon_{n-1}\varepsilon_n\,\lambda_{n-2}}{\varepsilon_{n+1}\,\varepsilon_{n+2}}. \tag{6}$$

Introduce scaled variables μ_i and per-m constants α_i:

$$\mu_i \;=\; \frac{\lambda_{n_i}}{\alpha_i\cos\theta}, \qquad \alpha_0 = \frac{1}{\varepsilon_{m+1}}, \qquad \alpha_{i+1} = \frac{\sigma_i}{\varepsilon_{n_i+1}\,\varepsilon_{n_i+2}\,\alpha_i}, \qquad \sigma_i = (-1)^i. \tag{7}$$

Then (6) becomes a recurrence with *unit* trailing coefficient — one FMA for the polynomial, one for the update:

$$\boxed{\mu_{i+1} = (a_i\,x + b_i)\,\mu_i + \mu_{i-1}}, \qquad a_i = \sigma_i\,\alpha_i^2, \qquad b_i = -a_i\,(\varepsilon_{n_i}^2 + \varepsilon_{n_i+1}^2), \tag{8}$$

with exact initial conditions $\mu_{-1} = 0$ and $\mu_0 = \lambda_{mm}$ (because $\lambda_{m+1,m} = \cos\theta\,\lambda_{mm}/\varepsilon_{m+1}$). The α_i carry the sign pattern $(+,+,-,-,+,+,\dots)$ of period four and $|\alpha_i| = \sqrt{|a_i|}$, which the implementation uses to reconstruct α from the stored a (§2.4). Note that $\cos\theta$ never appears in a denominator: at the equator the recursion runs on finite μ_i even though $\lambda_{n_i}(\pi/2) = 0$.

Both partial sums come out of the *same* μ sequence. Pre-fold the coefficients once per transform ($O(\ell_{\max}^2)$ total, embarrassingly parallel):

$$A_i = \alpha_i\,(\varepsilon_{n_i}\,a_{n_i-1,m} + \varepsilon_{n_i+1}\,a_{n_i+1,m}), \qquad B_i = \alpha_i\,a_{n_i,m}. \tag{9}$$

Using (4) and reindexing (all boundary terms vanish since $\lambda_{m-1,m} = 0$),

$$\sum_i \mu_i A_i = \frac{1}{\cos\theta}\sum_{\ell-m\ \text{even}} a_{\ell m}\,(\varepsilon_{\ell+1}\lambda_{\ell+1} + \varepsilon_\ell\lambda_{\ell-1}) = S_e, \qquad \cos\theta\sum_i \mu_i B_i = S_o. \tag{10}$$

Hence, per ring pair and per m,

$$p_1 = \sum_i \mu_i A_i, \quad p_2 = \sum_i \mu_i B_i, \qquad F_m^{\text{north}} = p_1 + \cos\theta\,p_2, \quad F_m^{\text{south}} = p_1 - \cos\theta\,p_2. \tag{11}$$

Cost: covering two ℓ's takes 2 real FMAs of recursion and 4 of accumulation ($p_{1,2}$ complex) — *3 flops per* $(\text{ring pair}, m, \ell)$, vs. 4–6 for the naive $\Delta\ell = 1$ recursion, with half the serial dependency chain.

2.3 Dynamic range: $2^{\pm 800}$ scaling

$\lambda_{mm} \propto \sin\theta^m$ underflows double precision catastrophically near the poles ($\sin\theta^{6000}$ at $N_{\text{side}} = 2048$ polar rings can be 10^{-10^4}). Two devices (both from `ducc0`):

- **Scaled representation.** A value is stored as (v, s) meaning $v\cdot 2^{800s}$ with $|v|$ kept below 2^{-60}; the seed $\sin\theta^m$ is computed by binary exponentiation in this representation (`mypow`, $O(\log m)$ multiplies — accurate to $\sim 10^{-15}$ where naive `pow` would lose m·ulp). While $s < 0$ the true value is $< 2^{-860}$ and contributes exactly 0 at double precision: the GPU thread advances the recursion *without accumulating* and re-scales ($v\cdot 2^{-800}$, s++) whenever $|v|$ exceeds 2^{-60}. Once $s = 0$ the thread switches permanently to the plain FMA loop (§2.4); normalised $\lambda_{\ell m}$ are bounded by $\sim \ell^{1/4}$ so no overflow re-scaling is ever needed there.

- m **cutoff.** On a ring with $\sin\theta$ small, all $m > m_{\rm lim}(\theta) \approx \ell_{\max}\sin\theta + \max(100, 0.01\,\ell_{\max})$ give F_m below double-precision relevance; those (ring, m) pairs are skipped entirely and their F_m set to 0 (identical rule to `ducc0`, so agreement with the reference is exact). This removes $\sim 30\%$ of the Legendre work.

2.4 GPU kernel design

CPU `ducc0` vectorises over rings within a SIMD register (AVX lanes share one coefficient stream) and threads over m. The GPU analogue maps **one thread to two adjacent (ring pair, m) chains**:

- Grid: y-dimension $= m \in [0, m_{\max}]$; x-dimension covers the $2N_{\rm side}$ ring pairs in blocks of 128 threads. At $N_{\rm side} = 2048$ this is $\sim 2\cdot 10^5$ blocks / $2.5\cdot 10^7$ threads, each running $(\ell_{\max} - m)/2$ recursion steps: ample parallelism, and the strongly m-dependent trip count is absorbed by the block scheduler.

- All threads of a block share one (A_i, B_i, a_i, b_i) stream (48 B per i): the loads are uniform across the block and hit L1/L2 broadcast; per-transform DRAM traffic from the tables is a single ~ 0.45 GB stream at $N_{\rm side} = 2048$ ($\sim$0.3 ms at HBM bandwidth). No shared memory is needed.

- Each thread keeps μ_i, μ_{i-1}, the four accumulators $\mathrm{Re/Im}(p_1, p_2)$, $x = \cos^2\theta$ and $\cos\theta$ in registers. The inner loop is unrolled two-fold like `ducc0`'s ((8) alternating the roles of μ_i/μ_{i-1}, avoiding a swap).

- **Two ring pairs per thread.** The μ recursion is a serial chain of two dependent FMAs per step ($\sim$8 cycles), against which each step offers only four independent accumulation FMAs — one chain per thread cannot hide the latency even at full occupancy (measured $\sim$28% of fp64 peak). Giving each thread two *adjacent* ring pairs (nearly equal θ, hence near-identical $m_{\rm lim}$ and scale-emergence points) doubles the independent FMA streams per thread and amortises the shared coefficient loads, the exact GPU analogue of `ducc0`'s SIMD-over-rings: measured $1.45\times$ on the Legendre stage (66 registers, no spills). The pair that surfaces earlier is advanced solo over the short gap, then both run in one fused loop.

- Warp divergence from the scaled start-up phase is mild: threads in a block are neighbouring rings, whose emergence ℓ differs little; the $m_{\rm lim}$ skip retires whole warps at once (rings are ordered pole→equator).

- The flop budget at $N_{\rm side} = 2048$, $\ell_{\max} = 6143$: $2N_{\rm side}\cdot\frac{1}{4}\ell_{\max}^2\cdot 12 \approx 3.7\cdot 10^{11}$ flops (before the $\sim$30% $m_{\rm lim}$ saving); A100 fp64 peak (non-tensor) is 9.7 Tflop/s.

The (a_i, b_i) table is built once per plan by a tiny kernel (one thread per m, sequential in i — the α recursion (7) is serial); the (A_i, B_i) pre-fold (9) runs per transform with one thread per (m, i), reconstructing $\alpha_i = \mathrm{sign}_4(i)\sqrt{|a_i|}$ and computing ε on the fly.

3 Stage 2: aliasing fold and per-ring inverse FFT

Ring values follow from (3). With $n \equiv n_\varphi$ pixels, $e^{im\varphi_j} = e^{im\varphi_0}e^{2\pi i\,jm/n}$ and $e^{2\pi i\,jm/n}$ depends only on $k = m \bmod n$: every F_m is *aliased* onto FFT bin k. At $\ell_{\max} = 3N_{\rm side} - 1$ this matters on *all*

rings, including the belt ($4N_{\rm side} < 2\ell_{\rm max}+1$). The fold kernel computes, per ring,

$$G_k \;=\; \sum_{m\equiv k \pmod n} t_m \;+\; \sum_{m\equiv -k \pmod n,\, m>0} t_m^*, \qquad t_m = F_m e^{im\varphi_0}, \tag{12}$$

(one thread per (ring, $m \le m_{\rm lim}$), `atomicAdd` into G; the two conjugate contributions make G Hermitian so the inverse DFT $f_j = \sum_k G_k e^{2\pi ijk/n}$ is real). $\mathrm{Im}\, F_0$ is discarded, matching `ducc0`.

Twiddle accuracy. $m\varphi_0$ can reach $\sim$5000 rad; naive `sincos` of the rounded product would cost $\sim m\cdot$ulp of phase. Since $\varphi_0 = \pi q/d$ with integers (q, d) (q=1, d=$4i'$ in the caps; $q \in \{0, 1\}$, d=$4N_{\rm side}$ in the belt), we reduce $mq \bmod 2d$ *in integer arithmetic* and call `sincospi` — the twiddle is then accurate to $\sim$1 ulp for every m. The same trick seeds the cap DFT rotations.

Belt rings all share $n = 4N_{\rm side}$ and are contiguous in the RING map layout, so a single batched cuFFT inverse c2c transform of shape ($2N_{\rm side}$+1, $4N_{\rm side}$) evaluates them; a trailing elementwise kernel writes $n\,\mathrm{Re}(\cdot)$ straight into the map segment.

Cap rings have $n = 4, 8, \ldots, 4(N_{\rm side}-1)$ — all different, awkward for cuFFT (thousands of tiny plans/launches). Two paths:

Small rings ($i < 64$; negligible pixel count): direct Hermitian inverse DFT,

$$f_j = G_0 + (-1)^j G_{n/2} + 2\sum_{k=1}^{n/2-1} \mathrm{Re}\Big(G_k\, e^{2\pi ijk/n}\Big), \tag{13}$$

one thread per pixel, rotating $e^{2\pi ijk/n}$ incrementally and re-seeding it exactly (integer-reduced `sincospi`) every 128 steps, which bounds the phase drift at $\sim 10^{-14}$. (Applied to *all* cap rings this kernel is $O(N_{\rm side}^3)$ and was $\sim$60% of the transform at $N_{\rm side} = 2048$; it survives only for the tiny rings where Bluestein padding would be wasteful.)

All other cap rings: Bluestein's algorithm through batched cuFFT. With chirps $w_t = e^{i\pi t^2/n}$ (computed exactly: $t^2 \bmod 2n$ in integers, then `sincospi`),

$$f_j \;=\; w_j \sum_k (G_k w_k)\; \bar{w}_{j-k}, \tag{14}$$

a linear convolution evaluated as a circular one of power-of-two length $M \ge 2n-1$. Rings are grouped into size classes $i \in [2^j, 2^{j+1})$ sharing $M = 2^{j+4}$, so *one* batched cuFFT pair (forward + inverse) per class handles a whole hemisphere pair population; the FFTs of the chirp kernels $\hat{b}$ are precomputed at plan build (715 MB at $N_{\rm side} = 2048$). This replaces the $O(N_{\rm side}^3)$ cap cost by $O(N_{\rm side}^2 \log N_{\rm side})$ and cut the $N_{\rm side} = 2048$ transform from 179 ms to 112 ms.

4 Accuracy

Correctness is defined as agreement with `ducc0` (`synthesis`, RING geometry from `Healpix_Base.sht_info()`). Because Almond implements the *same* recursion, coefficients, scaling thresholds and $m_{\rm lim}$ rule, agreement is at the level of floating-point noncommutativity, not method error. Gates in the test suite (random Gaussian $a_{\ell m}$, $\ell_{\rm max} = 3N_{\rm side} - 1$):

check	criterion	measured
NumPy reference vs `ducc0`, $N_{\rm side} \le 32$	$< 10^{-12}$	$\sim 10^{-14}$
GPU vs `ducc0`, $N_{\rm side} = 8 \ldots 1024$	$< 10^{-10}$	*(see §6)*
call-to-call determinism (atomics reorder)	$\le 10^{-13}$	pass

The transform is deterministic up to the order of the `atomicAdd`s in the fold stage, whose additions involve $\lesssim m_{\max}/n$ terms per bin; the observed call-to-call scatter is $\lesssim 10^{-15}$ relative.

A pure-NumPy reference implementation (`almond/reference.py`), ported line-by-line from `ducc0` and validated against it first, serves as the executable specification: every GPU kernel corresponds one-to-one to a block of that file, which made the CUDA port correct on first run.

5 Memory budget

Persistent device buffers of a plan at $(N_{\rm side}, \ell_{\max} = 3N_{\rm side} - 1)$, in double precision:

buffer	size	$N_{\rm side} = 2048$
coefficient table (a, b)	$16\,\mathrm{B} \cdot \ell_{\max}^2/4$	151 MB
pre-folded (A, B)	$32\,\mathrm{B} \cdot \ell_{\max}^2/4$	302 MB
phase F_m(ring)	$16\,\mathrm{B}\,(m_{\max}{+}1)N_{\rm ring}$	805 MB
FFT bins G	$16\,\mathrm{B}\,N_{\rm pix}$	805 MB
output map	$8\,\mathrm{B}\,N_{\rm pix}$	403 MB
Bluestein $\hat{b}$ tables	$\sim 16\,\mathrm{B} \cdot 43N_{\rm side}^2$	715 MB
total persistent		∼3.2 GB
transients (belt + Bluestein cuFFT)		∼1.8 GB

plus the input $a_{\ell m}$ (302 MB) if resident. Comfortable on a 40 GB A100 up to $N_{\rm side} = 4096$; the phase and G buffers could be eliminated altogether by fusing the fold into the Legendre kernel (§10).

6 Benchmarks

Measured on a dedicated Perlmutter GPU node (A100-SXM 40 GB; AMD EPYC 7763, 64 physical cores), SLURM job 55427184. Methodology: accuracy gate vs `ducc0` first; then median of ≥5–10 warm calls with `deviceSynchronize` around each; `ducc0` timed on the same node at 1/16/32/64 threads (best time reported); Almond timed both device-resident (the QML use case — the CG solver keeps data on the GPU) and including host→device→host copies of $a_{\ell m}$ and the map. Random Gaussian $a_{\ell m}$, $\ell_{\max} = 3N_{\rm side} - 1$, spin 0, single transform ($B = 1$).

$N_{\rm side}$	$\ell_{\max}$	acc. vs ducc	Alm dev. [s]	Alm host [s]	ducc 1t [s]	ducc best [s]	speedup	peak GB
128	383	2×10^{-13}	0.0003	0.0006	0.009	0.001 (32t)	2.6×	0.02
256	767	4×10^{-13}	0.0005	0.0014	0.053	0.002 (64t)	4.6×	0.08
512	1535	8×10^{-13}	0.0023	0.0060	0.353	0.013 (64t)	5.7×	0.31
1024	3071	3×10^{-12}	0.0133	0.0291	2.554	0.081 (64t)	6.1×	1.24
2048	6143	6×10^{-12}	0.0770	0.1558	19.110	0.479 (64t)	6.2×	4.97

Reading of the table:

- **Almond beats best-threaded (64-core) ducc0 at every resolution**, by 6.2× at $N_{\rm side} = 2048$ (77 ms vs 479 ms; 250× vs single-thread ducc0). Against the QML-relevant device-resident workload there is no CPU/GPU crossover to wait for — the GPU wins already at $N_{\rm side} = 128$.

- Including PCIe copies the win shrinks to 1.2–3.1×: transfers cost roughly as much as the transform itself at large $N_{\rm side}$ (455 MB round trip at $N_{\rm side} = 2048$). Batched/pipelined transfers or keeping data resident (SiMaster's mode) are essential.
- Whole-map throughput at $N_{\rm side} = 2048$: 50.3 Mpix/77 ms = 653 Mpix/s. Stage split: Legendre 65 ms (∼4 Tflop/s, 40% of A100 fp64 peak), Bluestein caps 7.6 ms, fold 4.9 ms, belt cuFFT 3.3 ms, pre-fold 0.6 ms.
- Accuracy vs ducc0 degrades slowly with $\ell_{\rm max}$ ($1.9 \times 10^{-13} \to 5.6 \times 10^{-12}$), consistent with floating-point noncommutativity of the fold atomics and the different twiddle paths, far below the 10^{-8} science gate.
- Peak device memory (plan + transients): 5.0 GB at $N_{\rm side} = 2048$, dominated by the phase/G buffers and Bluestein tables (§5); $N_{\rm side} = 4096$ fits a 40 GB A100.

6.1 High-throughput batched comparison

QML applies the SHT to many right-hand sides, and `ducc0`'s strongest many-transform configuration is not internal threading of one transform but its `ntrans` batch mode ($a_{\ell m}$ of shape $(B, 1, n_{\rm alm})$, one call, threads across the batch). On few-core runs this mode is dramatically better per column than intra-transform threading (login-node check: 426 → 41.6 ms/col from 1 to 16 threads at $N_{\rm side} = 512$). The honest throughput comparison is therefore per-column time at large B against `ducc0`-ntrans at 64 threads (same dedicated node; Almond runs `synthesis_device_batch`):

$N_{\rm side}$	B	acc.	Alm [ms/col]	Alm+copies	ducc0 64t [ms/col]	speedup
256	128	6×10^{-13}	0.39	1.55	2.11	5.4×
512	64	1×10^{-12}	1.98	6.52	12.01	6.1×
1024	64	4×10^{-12}	11.26	31.12	68.88	6.1×
2048	16	7×10^{-12}	76.96	150.27	460.96	6.0×

Two findings. First, on a full 64-core node `ducc0`'s batch mode is barely faster per column than its (well-implemented) intra-transform threading (461 vs 479 ms/col at $N_{\rm side} = 2048$): 64 concurrent CPU transforms saturate host memory bandwidth long before they saturate 64 cores. Almond's advantage in the throughput regime is therefore the same ∼6× as for single transforms, uniformly over $N_{\rm side} = 256$–2048 (1.4–3.1× if every batch crosses PCIe both ways).

Second, a negative result on the GPU side: a chunked Legendre kernel that shares one μ recursion among C columns ($4C$ accumulators per thread) *loses* to simply looping the single-transform pipeline (11.7 ms/col sequential vs 14.4–16.7 ms/col for $C = 2$–8 at $N_{\rm side} = 1024$): the accumulator registers destroy occupancy (150 registers at $C = 8$) and each column needs its own (A_i, B_i) loads, doubling the relative L1 pressure, while the amortised recursion was only ever 4 of 12 flops per step. The two-ring-pair kernel of §2.4 already owns the right trade: its two chains share one coefficient stream. Ring pairs beat column chunks on the GPU for the same reason SIMD-over-rings beats SIMD-over-columns on the CPU. The batched API therefore defaults to the looped single pipeline (asynchronous, no per-column synchronisation); the chunked kernel is retained behind `chunk>1` for future spin-2/adjoint experiments.

7 Design decisions and departures from ducc0

`ducc0` is aggressively CPU-shaped; porting it verbatim would waste the GPU. What was kept, and what was changed:

- **Kept: the mathematics.** Recursion (8), pre-fold (9), scaling thresholds, $m_{\rm lim}$ — bit-compatible behaviour makes validation trivial and inherits 15 years of numerical hardening.
- **Changed: parallel decomposition.** CPU: SIMD over ∼8–64 rings within one thread, OpenMP over m, ring-block cache tiling (`lstep`) to keep $a_{\ell m}$ resident in L2. GPU: one thread per (ring pair, m), $2.5 \cdot 10^7$-way parallelism, no tiling — the coefficient streams are block-uniform loads that L1/L2 serve at register speed, and the ∼150 MB tables are read once per transform from HBM.
- **Changed: coefficient handling.** `ducc0` recomputes the per-m coefficient vectors on each core inside the m loop (cheap on CPU, cache-resident). We precompute the full triangular (a, b) table once per plan (151 MB at $N_{\rm side}$=2048): GPU threads cannot afford a serial per-m preparation, and memory is cheaper than divergence. α_i is *not* stored — it is reconstructed as $\mathrm{sign}_4(i)\sqrt{|a_i|}$, saving 75 MB.
- **Changed: FFT stage.** `ducc0` runs pocketfft ring-by-ring inside the OpenMP loop with FFTPACK-style half-complex buffers. We split by ring population: one batched cuFFT for the $2N_{\rm side}$+1 equal-length belt rings, a direct Hermitian DFT kernel for the caps, and exact-rational `sincospi` twiddles instead of a cached `MultiExp` table.
- **Changed: precision management is per-thread scalar.** The SIMD-lane `corfac` machinery collapses to a simple three-phase scalar loop (skip / unscale / plain FMA), which is both simpler and divergence-friendly.

8 Adjoint synthesis and spin-2 (v0.2)

Adjoint ($Y^\dagger$: map→alm, ducc0's convention — the transpose under the alm inner product that counts $m > 0$ twice): per ring $\hat{G}_k = \sum_j f_j e^{-2\pi i j k/n}$ (cuFFT rfft on the belt, sign-flipped Bluestein on the caps), unfold $F'_m = e^{-im\varphi_0}\hat{G}_{m \bmod n}$ (with $\mathrm{Im}\, F'_0 \to 0$), then the Legendre adjoint $A'_i = \sum_{\rm pairs} \mu_i\,(F'_N + F'_S)$, $B'_i = \sum_{\rm pairs} \mu_i \cos\theta (F'_N - F'_S)$ and an α/ε-weighted gather to $a'_{\ell m}$. The reduction kernel gives each m one thread block owning *all* ring pairs (chunk loop), so global accumulation needs no atomics: per step, contributions are warp-shuffle-reduced (4 lanes-worth of chains per lane) and staged in per-warp shared tiles flushed every 128 steps. Validated against `ducc0.adjoint_synthesis` to $< 10^{-10}$ (nside 8–1024) and against Almond's own synthesis via the transpose identity to 10^{-12}. Cost: ∼1.5–1.9× the synthesis (147 vs 77 ms at $N_{\rm side}$=2048) — the shuffle-reduction overhead.

Spin-2: the native `ducc0` spin recursion (its spin-2-via-spin-0 shortcut with $1/\sin^2\theta$ ring weights is inexact near the poles; ducc itself reverts to the native path for $\sin\theta < 0.01$). Two scaled Wigner-d chains ($\lambda^\pm \sim d^\ell_{m,\mp 2}$) advance with a $\Delta\ell = 1$ three-term recursion in $\cos\theta$; eight accumulators build the $Q \pm iU$ combinations with parity-interleaved E/B coupling; initialisation uses half-angle power prefactors $\mathrm{prefac}(m)\, c^a s^b$ carried in the 2^{800}-scaled representation. Stage 2 (fold/FFT) is reused verbatim with the two components riding the batch-chunk axis. Synthesis and adjoint validated against ducc0 to $< 10^{-10}$ (nside 8–512).

Two porting bugs are worth recording. (i) $\mathrm{prefac}(m)$ underflowed to exactly zero for $m \gtrsim 514$: the factorial-table *ratios* need ducc's two-sided normalisation — one-sided clamping assumes a direction values do not actually keep. Same lesson as the `mypow` incident: port the invariant band, not the assumed monotonicity. (ii) the spin adjoint requires *conjugated* coefficients on the $\pm i$ cross terms (real-linear transpose: $y \mathrel{+}= c\,x \Rightarrow x' \mathrel{+}= \bar{c}\,y$). Both were found by mode-range bisection against ducc0.

Measured (dedicated A100-SXM + 64-core EPYC, job 55449334; device-resident, median of warm calls; ducc0 at 64 threads; accuracy gate passed in all cells):

$N_{\rm side}$	acc. (spin 2)	Almond [ms] (speedup vs ducc0 64t)			
		synth_0	adj_0	synth_2	adj_2
512	3×10^{-12}	2.0 (6.3×)	3.7 (3.3×)	9.0 (3.3×)	14.5 (1.8×)
1024	6×10^{-12}	12.1 (6.5×)	25.0 (3.2×)	49.4 (3.0×)	80.5 (1.7×)
2048	2×10^{-11}	71.9 (6.5×)	139.7 (3.2×)	363.5 (2.6×)	587.4 (1.6×)

The adjoint speedups (∼3.2× spin-0, ∼1.7× spin-2) trail the synthesis (6.5× / ∼3×) because ducc0's adjoint costs the same as its synthesis while Almond's reduction kernels pay the shuffle overhead — the known tuning target.

SiMaster integration: `almond.simaster.AlmondRealSHT` is a validated drop-in for `simaster.sht.RealSHT` (real coefficient basis, observed-pixel subsetting, batched columns; numpy or cupy in/out), tested against it for spin 0 and 2 on a cut sky to 10^{-10} together with the real-basis transpose identity.

9 v0.3: fused fold, tuning, SiMaster integration

The fold/unfold stages are now *fused into the Legendre kernels*: the forward extract scatters the twiddled F_m straight onto the ring FFT bins (`fold_scatter`, atomicAdd), and the adjoint init gathers F'_m from the half-spectrum $\hat{G}$ on the fly. The phase array no longer exists in any single-transform path (−805 MB spin-0 / −1.6 GB spin-2 at $N_{\rm side} = 2048$, plus one full memory pass in each direction); runtimes are unchanged within noise, as expected — this was a memory optimisation. The spin-2 adjoint reduction now runs four chains per lane (664→596 ms at $N_{\rm side} = 2048$; 168 registers, occupancy-limited).

SiMaster integration is complete short of the physics reruns: `almond` is an editable package and `CovModel`/`QMLWorkspace` accept `backend='almond'` (same `pure_callback` plumbing as `'ducc'`). The covariance operator $C \cdot x$ agrees between the two backends to 10^{-10} on a cut sky with mixed spin-0/spin-2 fields, and SiMaster's own test suite passes unchanged.

10 Roadmap

1. **val1/val2 QML validation** with `backend='almond'` (deliberately left to a SiMaster-focused follow-up).
2. **Spin-2 adjoint reduction restructure**: SADJ_PPT=4 is register-bound; a block-level two-stage reduction could recover the remaining ∼1.5× to parity with the forward kernel.
3. **Batched accumulator chunking** (one recursion feeding C columns) *measured to lose* to looping the single-transform pipeline (registers + L1 pressure; see §6.1).